

AWS Inventory, Graph & Agent

A thin slice of the dave.io product: connect to a customer's AWS account with a read-only role, ingest what is there and how it fits together, and put an agent on top that answers questions a DevOps engineer would act on.

PREPARED FOR	Tom Koren
PURPOSE	dave.io engineering assignment — presentation & defence
REPOSITORY	github.com/TomKoren1/dave.io_home-assignment
STACK	TypeScript · Fastify · Postgres · Neo4j · React Flow · Anthropic
VERIFIED AGAINST	A real AWS account (672299759593) and a mocked one (moto)

250

UNIT TESTS

21/21

AGENT EVALS · F1 1.0

14

DECISION RECORDS

37

LOGGED PROBLEMS

Contents

1 What was asked, and what was built

The brief · Scope delivered · How to demo it in ten minutes

2 Architecture

The pipeline · Why two databases · The domain model

3 The infrastructure half

Cross-account assume-role · Why the supplied IAM role was replaced · Partial failure

4 Deterministic analysis

The four analysers · Why verdicts are computed, not inferred · The traps

5 The agent

Sixteen curated tools · Citation validation · Remediation that is never applied

6 How correctness is established

Two eval tiers · What they caught · What they still cannot catch

7 The fourteen decisions

Each ADR, the alternative rejected, and the reason

8 The thirty-seven problems

Symptom, cause, fix and lesson for every one

9 Recurring failure patterns

The five shapes most of the bugs share

10 Limits, and what comes next

What breaks first at scale · Deliberate gaps · The next week of work

11 Interview preparation

Likely questions, with answers grounded in the code

1 What was asked, and what was built

The brief

Connect to a customer's AWS account using a read-only role, ingest the resources and the relationships between them into a store, put an LLM agent on top that answers questions about that account, and visualise the result in React Flow with a chat interface. Graded roughly equally across three axes — infrastructure, data engineering, and AI — with UX and production thinking on top. One hard rule: **the agent must never change anything**.

What is delivered

Axis	Delivered
Infrastructure	Cross-account <code>sts:AssumeRole</code> with an ExternalId, per-service and per-region scan units with independent failure, adaptive retry, SDK paginators, region discovery with a Resource Explorer fast path, and a replacement CloudFormation role that grants strictly less than the one supplied.
Data	Six collectors producing a typed resource model with explicit relationships; Postgres as the immutable system of record; Neo4j as a rebuildable graph projection; four deterministic analysers that compute security verdicts with evidence attached; scan diffing for change detection.
AI	A tool-calling agent over sixteen curated, parameterised tools; mechanical citation validation against the ARNs each tool actually returned; a read-only refusal enforced in code rather than prompted; and two eval tiers, one of which needs no API key and runs in CI.
UX	React Flow graph with dagre layout, streaming chat with per-tool progress, a findings sidebar, a changes tab, a trust panel, a runtime mock/real account toggle, and a step-by-step guide for connecting a real AWS account safely.

THE CLAIM WORTH MAKING

It runs end to end with **no AWS account and no cloud spend** — a mocked account seeded with deliberately non-obvious traps — and it has also been run against a real AWS account, where it found two genuine administrator users that an earlier version of the analysis had missed entirely. Both paths are the same code.

Demo in ten minutes

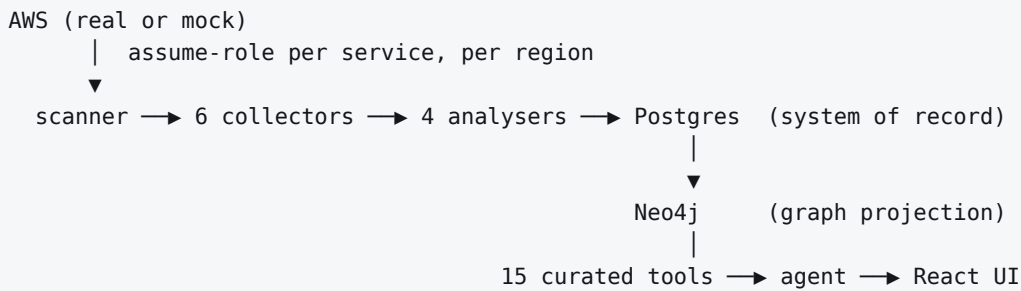
```
git clone <repo> && cd dave.io_home-assignment
cp .env.example .env           # add ANTHROPIC_API_KEY for the chat
docker compose up -d          # Postgres, Neo4j, moto
npm install

npm run seed                   # build the fictional customer account
npm run scan                   # discover it, persist it, project the graph
npm run dev:api & npm run dev:web
# open http://localhost:5173
```

The order to show things in, and the point each one makes:

1. **The graph.** Click a red node — a public bucket. The side panel shows the verdict and the *reason*: which policy statement, which access block.
2. **Ask "which S3 buckets are public?"** Two buckets carry byte-identical wildcard policies and only one is public. The answer names one and explains why the other is not. This is the single best demonstration in the project.
3. **Ask "what can reach the production database?"** It is `PubliclyAccessible: false` in a private subnet, and reachable by two distinct chains. Watch the tool calls stream as it works.
4. **Ask it to delete something.** It refuses, in code, not because the prompt asked nicely.
5. **Run `npm run drift` then scan again.** The Changes tab separates what matters from bookkeeping.
6. **Open the Trust panel.** Fifteen data checks run on demand in milliseconds with no API key, and the last agent eval run is shown in the product.

2 Architecture



Why two databases, and why that is not over-engineering

There is a strict hierarchy: **Postgres is authoritative, Neo4j is a rebuildable projection of it.** Each scan writes immutable snapshots to Postgres, then rebuilds the graph in one transaction. One writer, one direction of flow.

Neo4j earns its place because the brief's questions are overwhelmingly about relationships, and one of them — *what can reach the production database?* — is a variable-length path query. In Cypher that is one `MATCH` with a `*1..n` hop. In SQL it is a recursive CTE over a junction table that nobody will enjoy maintaining. The graph is the shape of the problem, not decoration.

Postgres earns its place because three things fit badly in a graph:

- **Scan history.** "What changed since the last scan?" needs immutable snapshots over time, not a mutable current-state graph.
- **Partial failure.** Per-(service, region) status, error codes, durations and API-call counts are a plain relational fact table.
- **Agent traces.** Conversations, tool calls and eval results are relational and high-volume, and must never compete with the queries the agent runs.

THE ANSWER TO "WHY NOT JUST ONE?"

The hierarchy is what makes two stores tolerable. If Neo4j is lost, or the graph model changes, it replays from Postgres with no rescan and no further AWS calls. That makes the graph *disposable*, and therefore safe to change — which is the property you want from the component whose schema you understand least well at the start.

The domain model

One `Resource` type across every service, carrying `arn`, `kind`, `name`, `region`, `accountId`, `tags`, raw `properties`, and a `derived` block. Analysers write only into `derived`, and every verdict is paired with a reason:

```
derived: {  
  isPublic, publicReason,           // anonymous access granted now  
  isUnprotected, unprotectedReason, // nothing would stop it being granted  
  isAdmin, adminReason,             // effective *.* on *.*  
  isIdle, idleReason, estimatedMonthlyCostUsd  
}
```

Relationships are explicit and typed — `IN_VPC`, `IN_SUBNET`, `HAS_POLICY`, `EXECUTES_AS`, `ATTACHED_TO`, `CAN_REACH` — and the internet is a first-class node (`arn:aws:daveio::internet`), so "reachable from the internet" is an ordinary graph traversal rather than a special case.

3 The infrastructure half

Cross-account access

The customer deploys one CloudFormation stack creating one IAM role. dave.io assumes it with `sts:AssumeRole` plus a per-customer **ExternalId**, which is the documented mitigation for the confused-deputy problem: knowing the role's ARN is not sufficient to assume it. Sessions are cached and renewed; role chaining caps at one hour, so the role asks for exactly that rather than pretending to more.

Why the supplied role was replaced

This is the sharpest thing in the brief, and probably deliberately so: the supplied template grants `ReadOnlyAccess` while the evaluation criteria ask whether candidates understand what read-only really means.

READONLYACCESS IS NOT READ-ONLY IN THE SENSE AN INVENTORY PRODUCT NEEDS

It grants roughly 7,000 actions including `s3:GetObject`, `dynamodb:GetItem`, `secretsmanager:GetSecretValue`, `ssm:GetParameter`, `kinesis:GetRecords` and `lambda:GetFunction` — which returns a presigned URL to the function's source. An inventory product needs to know that a bucket exists, who can reach it, and how it is configured. It never needs to read an object out of it. Granting the ability turns a compromise of dave.io's platform account into a compromise of every customer's *data*.

And `sqs:ReceiveMessage` is not even read-only literally. Receiving a message starts its visibility timeout and can hide it from the consumer that should have processed it. A scanner holding that permission can breach the brief's own hard rule through a permission nobody thinks of as a write. This is the detail to raise in the interview.

The replacement grants strictly less, and three things changed:

- **Permissions:** `SecurityAudit` + `job-function/ViewOnlyAccess`, plus an explicit `Deny` on data-plane reads. A `Deny` cannot be overridden by any `Allow`, including one a future AWS managed-policy update might introduce — which is what makes "we can see your infrastructure but not your data" a guarantee rather than a promise about today's policies.
- **Trust:** the scanner principal is named directly rather than `Principal: ...:root`. That `:root` does not mean "the root user" — it means every principal in the account that IAM permits to call `AssumeRole`. It is a common misreading, and worth flagging in the brief itself.
- **Attribution:** `sts:SetSourceIdentity` constrained to `daveio:*`, so the customer's own CloudTrail records which operator triggered a scan, immutably for the life of the session. A customer granting a third party standing access should not have to take our word for who did what.

The original template is kept alongside for comparison, and the role emits a `VerifyCommand` output so the customer can prove the `Deny` works from their own account with `aws iam simulate-principal-policy`.

Partial failure is the normal case

The unit of work is a `(service, region)` pair, and each one succeeds or fails independently with its own status, error, duration and API-call count. A scan of seventeen regions where `rds:eu-west-1` is throttled returns everything else and says precisely what is missing. The agent's system prompt carries the failed units, so it can warn that an answer may be incomplete without being asked. `SCAN_FAULT_INJECTION` exists to demonstrate this deterministically.

Region handling degrades the same way: Resource Explorer is tried as a fast path to learn which regions actually hold resources, falls back to `ec2:DescribeRegions`, and falls back again to a configured list. Every failure path widens the scan rather than narrowing it — the failure mode is "slower", never "silently incomplete".

4 Deterministic analysis

This is the decision that matters most in the whole project, and it is worth being able to state in one sentence: **questions like "which buckets are public?" are security reasoning, not data lookup, and the reasoning is done in code at ingest time — never by the model.**

A bucket is public if its policy or ACL grants a wildcard principal **and** neither the bucket-level nor the account-level public access block overrides it. That is a four-input boolean with a precedence rule. A language model can usually get it right, and "usually" is not a property you want in a security verdict. A unit test can get it right every time.

Analyser	Computes
Public access	<code>isPublic</code> — anonymous access is granted <i>now</i> — and separately <code>isUnprotected</code> — nothing would stop it being granted. Both with reasons.
Policy	<code>isAdmin</code> for roles <i>and</i> users, by reading policy documents rather than policy names, and attributing the grant to the specific managed or inline policy.
Reachability	<code>CAN_REACH</code> edges derived from security group rules, subnet routing and the internet node, so path queries traverse justified edges only.
Idle	<code>isIdle</code> from structural signals — attached to nothing, associated with nothing, stopped — with a list-price cost estimate for ranking.

PUBLIC VS UNPROTECTED — THE DISTINCTION TO LEAD WITH

Turning off S3 Block Public Access grants nobody anything. It removes the guardrail that would neutralise a permissive policy; an anonymous request still gets 403. Conflating the two is the most common misreading in this area, so they are separate verdicts, separate tabs, separate agent tools, and a paragraph of the system prompt — because the model conflates them too. Being able to explain this well is worth more than any feature in the project.

The traps in the mock account

The fixture is built to defeat a naive lookup. The answer key is written by hand in `topology.ts` rather than generated from the code under test, so an analyser bug cannot grade itself correct.

Trap	Why a naive answer fails
Two buckets, one policy	Byte-identical wildcard policies; only one is public. The other is neutralised by <code>RestrictPublicBuckets</code> . Naming both is the classic false positive.
A private database anyone can reach	<code>PubliclyAccessible: false</code> in a private subnet, reachable by two chains — three hops through the web and app tiers, two through a bastion with SSH open to the world.
A public database nobody can reach	<code>PubliclyAccessible: true</code> and its security group opens no ports. The obvious answer is wrong.
Admin hiding in plain sight	Three roles and two users are effectively admin. One grants <code>*:*</code> inline under the name <code>LegacyDeployRole</code> ; a user grants it inline under <code>BackupHelper</code> . Only findable by reading policy documents.
Money going nowhere	Unattached volumes, an unassociated elastic IP, a stopped instance, a NAT gateway in an abandoned region — about \$103/month.
A deliberate non-finding	<code>northwind-logs-archive</code> is genuinely unused, and is <i>not</i> in the answer key: proving it needs CloudWatch metrics the scanner does not collect, so reporting it would be a lucky guess rather than a finding.

5 The agent

A plain tool-calling loop, no framework

Ask the model, run whatever tools it requested, hand the results back, repeat until it answers in prose. Capped at eight iterations; tool results truncated at 12k characters. Every step is emitted as an event, so the UI shows *which tool is running* rather than a spinner — "what is the agent doing while it works" deserves a better answer than "Thinking...".

No framework, because the two decisions that actually define this agent — the tool boundary and citation validation — are precisely what a framework would hide behind its own abstractions, without removing any of the real work.

Sixteen curated tools, not text-to-Cypher

The model chooses which tool to call and with what arguments; it never writes the query. Text-to-Cypher was the obvious alternative and was rejected on four counts:

- **Safety.** A curated tool cannot express a mutation. The brief's hard rule becomes structural.
- **Correctness.** Hand-written Cypher for "every path from the internet to this resource" is reviewable, testable, and identical on every run.
- **Cost.** A tool call returns rows. Text-to-Cypher tends to return a schema, a failed query, an error, and a retry.
- **Auditability.** Because every tool records exactly which ARNs it returned, citations can be validated mechanically. This is the one that pays for the other three.

There is still a `graph_query` escape hatch for genuinely novel questions. It runs behind a lexical write-clause validator **and** inside a Neo4j read transaction, because neither layer is trusted alone.

Citation validation

THE MECHANISM, IN ONE PARAGRAPH

Every tool records the ARNs it returned. Every ARN appearing in an answer is checked against that set, and anything unsupported is flagged on the response and shown to the user. This turns the most dangerous failure mode — a confident, plausible, invented identifier — from something a prompt hopes to prevent into something the system detects. Any unsupported citation fails an eval case outright.

It has earned its place twice. Once it cried wolf at a Markdown backtick and needed the ARN pattern fixed (log #12). Once it caught the agent emitting `arn:aws:ec2:...:instance/` — a truncated placeholder — and the root cause turned out to be a tool that did not return the resource its own question was about (log #32).

"Never change anything", enforced at four layers

1. The IAM role has no write permissions and an explicit Deny on data reads.
2. The scanner only ever calls `Describe` / `List` / `Get`.
3. No tool can express a mutation.
4. Raw Cypher is lexically validated and run inside a read transaction.

And a fifth, added because it had to be: when a user asks the agent to change something, the answer must explicitly say it cannot. Three attempts to achieve that by prompting failed — one of them because the prompt's own Style rules ("do not pad with caveats") were competing with the constraint. The guarantee now lives in code, in `readOnlyGuard.ts`, applied after the model has spoken (log #15, ADR-009).

THE GENERALISABLE POINT

If a property must hold, do not ask a model to hold it. Three prompt attempts, each plausible, each failing differently, cost more than the twenty lines that made it structural — and the twenty lines cannot regress when the prompt is edited for an unrelated reason.

Remediation: the fix, written out, and never applied

Every finding carries the exact commands that would resolve it, what each might break, and a read-only command to confirm it worked. There is no endpoint that runs them. The absence of an *Apply* button is the feature — see ADR-014 — and the commands are computed from the same evidence as the verdict, so they target the inline policy that actually grants `*:*` rather than the `AdministratorAccess` a model would reach for. The agent has a tool for it and is told to quote the result verbatim rather than compose its own.

One honest boundary

Neo4j Community has no role-based access control, so a genuinely read-only database *user* is not available. The read transaction is enforced at the driver, not the database. In production this would be an Enterprise read-only role or a read replica. Recorded in log #7 rather than glossed over — and worth volunteering in the interview, since it is the kind of thing a good reviewer will find.

6 How correctness is established

Two eval suites that fail for different reasons, which is the point of splitting them.

Tier 1 — ground truth over the data

Seeds the mock account, runs a real scan, and asserts the result against the hand-written answer key. **No model, no API key, about two seconds**, and it runs in CI on every commit. Fifteen checks, including the ones that matter most: the neutralised bucket is *not* public, the inline-admin role *is* admin, the private database is reachable by both expected chains, and the publicly-flagged database is reachable by none.

Tier 2 — answer quality

Twenty-one cases against the live agent, scored on the ARNs each answer cites, with precision and recall. Each case asserts what must be cited, what must **not** be (the traps), and which tools should have been chosen.

Last recorded full run on `claude-sonnet-5`: **21/21, mean F1 1.0, no unsupported citations**.

Precision matters as much as recall precisely because of the traps: an answer naming every bucket achieves perfect recall and is useless.

GETTING THERE IS THE BETTER ADVERTISEMENT THAN THE SCORE

The suite caught four real problems, and two of them were problems with the tests:

- The agent answering "please delete this volume" with the volume's details and a CLI command — safe, useful, and never saying it could not act (log #15).
- A defect in a test rather than an answer (log #13) — and two days later that same relaxed test turned out to have been hiding a real tool defect (log #32).
- Two attempts to test a *negated* inference with a forbidden-substring pattern, both of which failed correct answers. The second failed on "a different kind of risk than an unused admin role" — a sentence stating the very distinction being tested for.

What this does not catch, stated plainly

An answer that cites exactly the right resources and describes them wrongly. The `mustMention` patterns cover the cases where that has teeth, but a genuinely adversarial wrong answer with correct citations would pass. Closing it needs an LLM judge over a larger case set. Relatedly, lexical patterns cannot test a negated claim at all — "X is unused" and "X is not unused" differ by one token that carries the whole meaning.

How you would know a change made it worse

Tier 1 fails in CI within seconds. Tier 2 produces a mean F1 and a pass count, stored in `eval_runs` and written to `evals/results/` so two runs can be diffed. The split makes failures

diagnosable: if tier 1 passes and tier 2 fails, the data is right and the agent misused it — a prompt or tool-description problem. If tier 1 fails, nothing about the agent is worth looking at yet.

CI

Three jobs: a static job (format, typecheck, 176 unit tests), an integration job that stands up Postgres, Neo4j and moto and runs the tier-1 suite, and a secret scan. The secret scan pairs gitleaks with a deterministic `git ls-files` check that no environment file is ever tracked — because a probabilistic check that silently matches nothing is worse than no check at all, which is exactly what happened in log #27.

7 The fourteen decisions

Full records with alternatives and consequences in `docs/DECISIONS.md`.

ADR-001 — TypeScript across the whole repo

One language for the scanner, the API, the agent and the UI. The domain model is defined once in `packages/shared` and imported by everything, so a change to a resource shape is a compile error rather than a runtime surprise at the API boundary. Python would have been better for nothing here that TypeScript is not adequate for.

ADR-002 — moto in Docker as the mock AWS control plane

`moto` implements the real AWS APIs, so the scanner is written against the genuine SDK and does not know it is talking to a mock — the same code runs against real AWS with only an endpoint override removed. LocalStack's community edition cannot do RDS, which this project needs. The cost is two capability gaps, both recorded rather than hidden: no pagination simulation, and no Resource Explorer.

ADR-003 — Postgres as system of record, Neo4j as derived projection

Covered in section 2. The load-bearing part is the hierarchy, not the pair.

ADR-004 — Security reasoning is computed in code, never by the model

Covered in section 4. This is the decision to lead with if asked "what would you defend hardest?"

ADR-005 — A curated tool library instead of text-to-Cypher

Covered in section 5, with the four reasons.

ADR-006 — Citations are validated mechanically

The one countermeasure against invented identifiers that does not depend on the model cooperating. It requires the tool boundary of ADR-005 to work at all, which is why the two decisions are really one.

ADR-007 — Replacing the supplied read-only role

Covered in section 3, with the `sqs:ReceiveMessage` detail.

ADR-008 — Two tiers of evaluation

Tier 1 is deterministic, free and fast enough for CI; tier 2 costs money and time and needs a key. Keeping them separate means the expensive suite is not the gate, and a failure in either one localises the problem immediately.

ADR-009 — The read-only refusal is enforced in code, not prompted

Covered in section 5, including the two-instructions-fighting diagnosis.

ADR-010 — Onboarding is guided in the UI, not performed by it

The obvious feature is a form: paste a role ARN and an ExternalId, press connect. This API has no authentication, so that form would be an unauthenticated endpoint that assumes a role into somebody's AWS account and stores a value the role template itself calls a credential. Instead the server generates what a customer needs, renders the exact command, and tests the connection that is already configured — so putting the role ARN into configuration stays a deliberate act by an operator with host access. **Building it would have made the demo look more complete and the product less defensible.**

ADR-011 — Evaluation results are shown in the product

The Trust panel shows what is checked and when it last ran. A tool that makes security claims should be able to show its own working to the person relying on it, not only to the engineer who wrote it.

ADR-012 — "Public" and "unprotected" are separate verdicts

Covered in section 4. Prompted by a real question about a real bucket with Block Public Access disabled: the agent was correct to say it was not public, and the product was wrong to have nowhere to say it was unguarded.

ADR-014 — Remediation is generated and never applied

Every finding carries the exact commands that would fix it, what each one might break, and a read-only command to confirm it worked — as **strings**. There is no endpoint that executes them and no credential that could.

The obvious next step is a *Fix it* button, and it is the one thing this product must not have: it would undo ADR-005, ADR-007 and ADR-009 in a single click, buying convenience and selling the only property that makes standing access to a customer's account defensible. The honest version is also more useful — the person who knows whether a public bucket is a mistake or a deliberate CDN origin is at the keyboard, not in the scanner. So the product removes the error-prone part, working out the precise command, and leaves the judgement where the knowledge is.

Computed, not generated, by the argument in ADR-004. The fixture contains a role whose admin comes from an inline policy named `legacy-deploy-inline`; a model reaching for the obvious `detach-role-policy --policy-arn .../AdministratorAccess` would emit a command that runs cleanly and fixes nothing. And `caution` is a *required* field: an unprotected bucket rates **low** because no anonymous access exists to lose, while a genuinely public one rates high — ADR-012 carried to the last step. In the UI the caution renders *above* the copy button, because below it, it is read second.

ADR-013 — The account toggle switches at runtime and never writes .env

Switching accounts from the UI must not rewrite configuration — a restart should always return to a known state. Switching also deliberately does not rescan: the graph keeps showing the previous account's inventory until one runs, and the banner says so, rather than letting someone read one account's resources under another's name.

8 The thirty-seven problems

Full write-ups in `docs/ENGINEERING-LOG.md`. Each entry below gives the symptom, the cause, the fix, and what it generalises to.

Mock-environment capability gaps

#1 `AttachRolePolicy` fails for AWS-managed policies under moto

CAUSE	moto does not load AWS-managed policies unless told to; <code>AdministratorAccess</code> simply does not exist, so attaching it throws <code>NoSuchEntity</code> .
FIX	<code>MOTO_IAM_LOAD_MANAGED_POLICIES=true</code> in compose.
LESSON	A mock's defaults are a product decision, not a bug. Read them before assuming your code is wrong.

#2 Resources silently vanishing into a second AWS account

CAUSE	moto namespaces resources by the account id in the credentials. Assuming a role whose ARN names a different account than <code>MOTO_ACCOUNT_ID</code> puts resources in a separate namespace — seeds succeed, scans find nothing.
FIX	Keep the role ARN's account equal to the mock account id; documented in <code>.env.example</code> .
LESSON	The first of <i>three</i> bugs in this project caused by two sources of truth for "which account am I looking at?". See #17 and #31.

#3 moto does not simulate pagination

CAUSE	Thirty instances with <code>pageSize: 5</code> returned one page, so paginator handling was untested by construction.
FIX	Unit-test paginators against a stubbed client rather than the mock.
LESSON	Know which properties your fixture <i>cannot</i> exercise, and test those elsewhere — a green suite over a mock that cannot fail is not evidence.

#4 AWS Resource Explorer is not available in the mock

CAUSE	moto returns an HTML 404 for <code>resource-explorer-2</code> .
FIX	Treat it as an optional fast path with SDK enumeration as the tested fallback; the degradation path is what gets exercised locally.
LESSON	It also cannot be relied on in production: the index must be created in the customer account, which a read-only role cannot do. Worth raising as feedback on the brief.

Type and tooling traps

#5 `InstanceType` is an enum, not a string

- CAUSE The SDK types it as a union of literals; a plain `string` will not assign.
- FIX Use the SDK's own `_InstanceType` type in the seeder.
-

#6 The SDK paginator refuses a hand-rolled fake client

- CAUSE Paginators check `instanceof` the real client class.
- FIX Construct a real client and stub only its `send`.
- LESSON Fake at the narrowest seam the library will accept, not the widest one that compiles.
-

#9 An SDK type silently degraded to `any`, and `skipLibCheck` hid why

- CAUSE Anthropic SDK 0.33.1 had broken self-referencing imports. `Message` resolved to `any`, so a type predicate degraded and its callback parameter became implicitly `any` — surfacing as an unrelated `TS7006`.
- FIX Diagnosed by assigning the suspect type to a `number` and watching it succeed. Upgraded to 0.128.0 and aliased the content-block types once so it cannot silently return.
- LESSON `skipLibCheck` buys build speed by hiding exactly this. If a type error makes no sense, test whether the type is `any`.
-

#10 Three versions of Vite in one workspace

- CAUSE Transitive version drift across workspaces.
- FIX Clean reinstall, aligned on one version.
-

#14 Vite does not see the repo-root `.env`

- CAUSE Vite loads `.env` relative to its own root, not the monorepo root.
- FIX Point `envDir` at the repo root so one file configures everything.
-

#18 Prettier silently deleted 47 lines of a CloudFormation template

- CAUSE Prettier's YAML formatter reflows folded block scalars (`Description: >`) and dropped content from one.
- FIX `infra/*.yaml` in `.prettierignore`.
- LESSON It happened a second time from an unidentified cause, which is why #22 exists.
-

Database and query behaviour

#8 `LIMIT $limit` rejected with "found 100.0"

CAUSE	JavaScript numbers reach Neo4j as floats; <code>LIMIT</code> requires an integer.
FIX	<code>neo4j.int()</code> inside the shared <code>clamp()</code> helper, so every query is fixed at once.
LESSON	Fix a type-boundary bug in the one place the boundary is crossed, never at each call site.

#7 Neo4j Community cannot enforce a read-only database user

CAUSE	RBAC is an Enterprise feature.
FIX	Enforce read-only at the driver with explicit read transactions, plus a lexical write-clause validator; record the residual gap honestly.
LESSON	When you cannot have the guarantee you want, name the one you actually have. Volunteer this in the interview.

#11 A diff field that arrived with no `before` key

CAUSE	Absent and <code>null</code> were conflated when diffing snapshots.
FIX	Distinguish them explicitly in the diff model.
LESSON	"Missing" and "empty" are different facts. This project made the same class of mistake three more times — #28, #30 — with empty strings.

Agent and evaluation

#12 The citation validator cried wolf at Markdown

CAUSE	The ARN regex swallowed a trailing backtick, so a correct answer was flagged as citing an unsupported ARN.
FIX	Exclude Markdown punctuation and strip trailing noise — but <i>not</i> the colon, since RDS and SNS ARNs contain colons in the resource section. The first fix over-corrected and truncated RDS ARNs instead.
LESSON	An indicator is only as good as its false-positive rate. A validator that flags correct answers gets ignored, which is worse than not having one.

#13 An eval failure that was a bad test

CAUSE	<code>blast-radius</code> expected the agent to cite <code>prod-web-1</code> , but <code>find_reachable_from</code> did not return the source, so its ARN was not citable.
FIX	At the time: relaxed the assertion. That was the wrong call — see #32.
LESSON	"The test is wrong" and "the system is wrong" are both live hypotheses, and I chose the first too quickly.

#15 The refusal the model would not make, and two instructions fighting

CAUSE	Asked to delete a volume, the agent returned the volume's details and a CLI command — safe and useful, but never saying it could not act. Three prompt fixes failed; one failed because the Style section's "do not pad with caveats" was competing with the constraint.
FIX	Enforced in code after the model has spoken (<code>readOnlyGuard.ts</code> , ADR-009).
LESSON	Prompt instructions compete, and the loser is not predictable. If a property must hold, make it structural.

#16 Change detection looked broken because the fixture had been re-seeded

CAUSE	Re-seeding regenerates resource ids, so every resource read as removed and re-added.
FIX	Documented the seed → scan → drift → scan order; the diff was correct throughout.
LESSON	Before debugging a diff, check whether the identities being compared are stable.

#25 The eval CLI would have run sixteen fixture questions at a real account

CAUSE	The tier-2 runner had no mode guard, so it would happily ask mock-specific questions of a real account.
FIX	Refuse to run unless the mode is mock.

The configuration failures — the most serious class

THREE ROUTES TO THE SAME ARTEFACT

#2, #17 and #31 are the same bug three times: **two sources of truth for "which account am I looking at?"** — the endpoint the calls go to, and the ARN the results are labelled with — with nothing asserting they agree. The output is a complete, confident, entirely fictional inventory of a real AWS account, sitting in the database looking exactly like a genuine scan. It is the worst thing this system can produce.

#17 A real-account scan that silently inventoried the mock

CAUSE	<code>AWS_ENDPOINT_URL</code> left in <code>.env</code> for the mock is read <i>directly by the AWS SDK</i> from the environment. It is an SDK-wide override, so it redirected every client — STS included — away from AWS regardless of <code>AWS_MODE</code> . The scan enumerated moto and labelled the results with the real account id from the role ARN. The connection test passed , because moto accepts any AssumeRole it is given.
FIX	<code>config.ts</code> deletes every SDK endpoint override from <code>process.env</code> unconditionally at import, warns loudly, and passes the mock endpoint explicitly instead. Placeholder credentials that would shadow real ones are removed the same way.
LESSON	A config value your dependency reads directly from the environment is not yours to interpret. The only fix that works is deleting it.

#31 A mock scan recorded under a real AWS account id

CAUSE	<code>AWS_MODE=mock npm run scan</code> against a <code>.env</code> pointing at a real account. The mode override made <code>configuredMode</code> "mock", so the mock branch honoured <code>AWS_TARGET_ROLE_ARN</code> (which #24 had deliberately made it do) — but that ARN named the real account while the endpoint override sent every call to moto. 100 mock resources were persisted under a real twelve-digit account.
FIX	The test for "does <code>.env</code> describe the mock?" is now the account id , not the mode flag. Extracted as a pure exported function with ten tests, because the condition was load-bearing and untestable inside a module that reads <code>process.env</code> at import.
LESSON	When two configuration values must be consistent, do not document the requirement — compute one from the other, or refuse.

#21 A configuration knob that existed only in code

CAUSE	<code>AWS_SCAN_REGIONS</code> had a code default but was never documented in <code>.env.example</code> , so a real account scanned three regions instead of seventeen.
FIX	Documented, with the blank-means-discover semantics made explicit.

#23 The test suite scanned a real AWS account

CAUSE	The ground-truth suite seeds a fixture and scans it. With <code>.env</code> in real mode it would do that to a real account.
FIX	An <code>isMock()</code> guard that refuses to run. The first attempt wrote <code>!isMock</code> — a function reference, always truthy, so the negation was always false and the guard was silently disabled.
LESSON	A guard that cannot fire is worse than no guard, because it is trusted. Test that your guard fires.

#24 The toggle quietly disabled the project's own onboarding variables

CAUSE	Adding the runtime account toggle made mock mode derive its role ARN and ExternalId, so editing <code>AWS_TARGET_ROLE_ARN</code> — the variable the project ships in <code>.env.example</code> — appeared to do nothing.
FIX	Honour them whenever <code>.env</code> itself describes the mock. (Which then needed #31's account-id refinement to be safe.)
LESSON	A feature that makes documented configuration inert is a regression even if nothing fails.

The IAM template

#20 The IAM template had never been deployed, and did not work

CAUSE	It referenced <code>arn:aws:iam::aws:policy/ViewOnlyAccess</code> , which does not exist. <code>ViewOnlyAccess</code> is a <i>job-function</i> policy and lives under <code>job-function/SecurityAudit</code> on the line above <i>is</i> at the root, which is what makes it easy to get wrong. CloudFormation only says so at deploy time; the stack rolled back and had to be deleted before retrying.
FIX	Corrected path, plus tests asserting it.
LESSON	An artefact nothing executes is an artefact nothing checks. This one was described in detail in the README and had never once been run.

#22 The template lost its reasoning twice, so now a test holds it

CAUSE	~70 lines of rationale vanished twice — once demonstrably from Prettier (#18), once from a cause never identified.
FIX	Seventeen guard tests asserting both the permissions <i>and</i> the prose.
LESSON	A document nobody checks rots. Content guards look odd in most repositories; here the content had been silently deleted twice.

#19 The onboarding guide taught the mistake it was meant to prevent

CAUSE	Two ARNs are involved — the scanner principal (an input) and the created role (the output) — and the guide did not distinguish them clearly enough.
FIX	Named both roles explicitly in the guide and the template header.
LESSON	Prose was not enough; #28 shows the same confusion recurring and being fixed in code.

Secrets

#26 A live API key reached git history, and the secret scan passed

CAUSE	A file named <code>.env.mock-backup</code> , created as a convenience step during unrelated work, was committed with a live Anthropic key and the customer ExternalId. <code>.gitignore</code> listed exact filenames; gitleaks' rules did not match.
FIX	Key revoked and rotated. History rewritten with <code>git filter-branch</code> and force-pushed with a lease; verified no object in history contains the key. <code>.gitignore</code> widened to <code>.env*</code> patterns. A deterministic CI check added: <i>no environment files may be tracked</i> , whatever they are called.
LESSON	Two of them. Pair a probabilistic check with a deterministic one covering the common case. And the riskiest artefacts are the incidental ones — this file was not part of any feature, and nobody reviews a convenience step.

#27 The secret scan I added to catch #26 was not running

CAUSE	The new rule used a negative lookahead to exclude placeholders. gitleaks is written in Go, and RE2 has no lookarounds by design — so gitleaks panicked at startup and scanned nothing . It failed loudly only by luck, because a later step could not find a report file.
FIX	Exclusions moved to a rule allowlist, which is the mechanism provided for it.
LESSON	This is #26 one level up. There, green meant "my rules did not match". Here it would have meant "my rules did not load" — strictly worse, because it is trusted. The deterministic <code>git ls-files</code> guard needs no rules and no regex engine, and would have caught the original leak alone.

Found by real use, at the end

#28 Four bugs in the onboarding path, found by someone actually using it

SYMPTOM	A real deployment of the role template, followed by the UI's own connection guide. Four separate failures, none reachable from any unit test.
CAUSES	(1) CloudFormation caps <code>Description</code> at 1024 characters; the explanatory header had reached 3,952, so the template could not be deployed at all — and the guards from #18/#22 asserted the prose was <i>present</i> , never that the file still <i>worked</i> . (2) The guide pre-filled the deploy command from <code>sts:GetCallerIdentity</code> verbatim. That reports a <i>session</i> , so it returns <code>arn:aws:sts:...;</code> a trust policy needs the <code>arn:aws:iam:...;</code> identity behind it. Against the mock it offered <i>moto's own identity</i> as the principal to trust in a real account. (3) <code>AWS_TARGET_ROLE_ARN</code> held a user ARN — the correct answer to the question asked two steps earlier, and something <code>sts:AssumeRole</code> can never assume. (4) <code>AWS_REGION=</code> was blank; Zod's <code>.default()</code> fires only on <code>undefined</code> , and a blank <code>.env</code> line parses as <code>""</code> , so every AWS client was built with an empty region.
FIXES	Prose moved to <code>#</code> comments (no length limit, stripped before evaluation) with guards on the folded length; a tested <code>principal.ts</code> that converts session ARNs to principal ARNs and returns a <i>reason</i> rather than a guess where there is none; <code>validateAssumeRoleTarget()</code> naming the two-role mix-up; and a <code>blankAsUnset</code> wrapper — deliberately <i>not</i> applied to the two variables where blank is documented and meaningful.
LESSON	Three of the four were in code whose job is to <i>explain</i> something, and explanatory code fails silently because nothing downstream consumes it. A wrong ARN in a code path throws; a wrong ARN in an instruction gets pasted. The fix for all of it was the same: make the thing that explains also be the thing that computes.

#29 "No admin roles" was true, and read as "nobody has admin"

SYMPTOM	A scan of a real account returned a completely empty findings panel — including no administrators — for an account where two of three IAM users hold <code>AdministratorAccess</code> .
CAUSE	The collector fetched policy documents for roles only. The pipeline loop, commented "which principals are administrators?", filtered to <code>IamRole</code> . The analyser was never the problem: it reads policy documents, identically shaped for both.
WHY NOTHING CAUGHT IT	The mock account contained no IAM users at all. The fixture shared the blind spot , so 157 passing tests said this worked.
FIX	Collect user policies (sharing the managed-policy cache); accept both kinds; match both labels in the query; add three users to the fixture — one admin by managed policy, one admin inline under the name <code>BackupHelper</code> , one scoped as the negative class — and three ground-truth checks. Reverting only the pipeline filter fails two of them.
LESSON	It did not say "I do not check users". It said "No role grants unrestricted access" beside an empty panel, which reads as a clean bill of health. A gap that presents as reassurance is the one kind of false negative a security tool cannot ship.

#30 An idle-detection rule that was correct and never fired once

SYMPTOM	The README listed an unassociated elastic IP among the mock's waste. The findings never contained one.
CAUSE	moto reports an unassociated address as <code>InstanceId: ""</code> — an empty string, not an absent key. The collector's <code>?? null</code> only substitutes for <code>undefined</code> , and the rule asked <code>!== null</code> . So <code>"" !== null</code> was true and every orphaned address read as in use.
WHY NOTHING CAUGHT IT	The answer key did not list the address either. Fixture and code shared the omission — the same shape as #29, two entries apart.
FIX	<code>absentIfBlank()</code> at the collector boundary; a presence test in the analyser as a second layer; the address tagged in the fixture so the answer key can name it; six unit tests stating the empty-string case explicitly.
LESSON	Two bugs of this shape in two days argues for a rule: normalise "absent" where data enters, once, and never ask <code>!== null</code> about a string from an external API.

#32 The validator caught the agent inventing an ARN, and the tool was at fault

SYMPTOM	<code>cited unsupported ARNs: arn:aws:ec2:us-east-1:...:instance/</code> — a truncated prefix, followed in the prose by "let me confirm exact ARN".
CAUSE	<code>find_reachable_from</code> returned only the <i>targets</i> . The question is "if prod-web-1 were compromised, what could it reach?", so every answer names prod-web-1 — whose ARN was in no tool result and therefore not citable.
THE UNCOMFORTABLE PART	#13 recorded this same case failing and concluded the <i>test</i> was wrong. Relaxing it made the suite green and left a tool whose result set cannot support the obvious answer to its own question. The defect waited two days and returned as a fabricated identifier.
FIX	The tool returns the source at <code>hops: 0</code> . The separate lookup also distinguishes "no such resource" from "reaches nothing", which the single query had conflated.
LESSON	The tell I missed: I relaxed an assertion and wrote a comment justifying it in terms of a <i>system limitation</i> . That comment was a defect report filed in the wrong place. A test relaxed because the system cannot satisfy it should open an issue, not close one.

Found by using it, not testing it

#33 Two test expectations that were wrong about the shell

SYMPTOM	Two of the first remediation tests failed, and both times the implementation was right.
CAUSE	<code>shellQuote("it's")</code> produced <code>'it\'\'s'</code> and the test expected <code>'it\'s'</code> . The intuitive form is the wrong one: a POSIX single-quoted string has no escape character, so a quote cannot be backslash-escaped inside it — the quoting must close, emit an escaped quote outside, and reopen.
FIX	Settled by running <code>printf '%s\n' 'it\'\'s'</code> in a real shell rather than arguing from memory. Had I "fixed" the implementation to match the test, every remediation for a resource with an apostrophe in its name would have emitted a command that fails on paste — with green tests.
LESSON	When a test and the code disagree about how an <i>external system</i> behaves, neither is evidence. Go and ask the external system. #13 and #32 both went wrong by reasoning about the disagreement instead of resolving it.

#34 Three UI bugs that only existed on screen

CONTEXT	For most of this project there was no browser on the build host, so "the frontend works" meant it compiled, served and returned correct JSON. Headless Chrome went in to render this document; pointing it at the running app found three bugs in the first two screenshots.
CAUSES	(1) Six findings tabs in a 288px sidebar, laid out <code>flex-1</code> — which does not shrink them, because flex items default to <code>min-width: auto</code> and overflow instead. They spilled across the graph pane where the floating Filter button at <code>z-10</code> covered two of them, making part of the primary navigation unreachable. (2) The copy button was positioned absolutely inside the <code><pre></code> with padding reserved — fine in the wide modal it was built for, broken in remediation's 320px panel where the block scrolls horizontally and the padding scrolls away with it, leaving the button sitting on the command. (3) A 404 on every page load for a missing favicon.
LESSON	All three are invisible to every test in the repository and would have survived any number of green runs — and one of them hid part of the main navigation. The gap was not rigour, it was never having looked. Where the output is visual, look at it; where it is a command someone will run, run it.

#35 A documented command that did not exist

SYMPTOM	<code>npm run query -w @daveio/api</code> , documented in the README's command table, failed: no such script.
CAUSE	The CLI file existed, worked, and its own header comment gave that exact invocation. The npm script was simply never added — so the tool had been written, documented in two places, and was unreachable by the command both of them named.
FIX	The script, plus a guard of the same shape as the template tests: <code>readme.test.ts</code> extracts every <code>npm run ...</code> the README tells you to type and asserts it exists, then checks the counts it quotes — ADRs, agent tools, eval cases, tier-1 checks — against the files they describe. Verified to fire rather than pass vacuously.
LESSON	The third documentation failure here that nothing could catch (see #20, #22). Instructions are an interface, and an interface with no tests drifts. Eight lines of regex replaced two full manual audits — both of which I had already done, and both of which missed this.

#36 `.env` was never loaded on Windows, and the defaults hid it

SYMPTOM	Reported from a Windows laptop: a real key in <code>.env</code> , and <code>"agent": "ANTHROPIC_API_KEY not set"</code> . Everything else worked — Postgres, Neo4j, the scan, the graph.
CAUSE	Four files built a filesystem path with <code>new URL(..., import.meta.url).pathname</code> . That is a <i>URL</i> path: on Windows it yields <code>/C:/projects/app/.env</code> , and on any OS it percent-encodes a directory containing a space. <code>fs</code> opens neither, and <code>dotenv</code> 's <code>quiet: true</code> made the <code>ENOENT</code> silent — so not one variable from <code>.env</code> was ever loaded.
WHY ONLY CHAT BROKE	The Zod defaults happen to match <code>.env.example</code> , so everything else ran on defaults that coincidentally agreed. <code>ANTHROPIC_API_KEY</code> is the only variable with no default. Every other edited value — ports, passwords, <code>AWS_MODE=real</code> , scan regions — was ignored just as completely with no symptom at all.
FIX	<code>fileURLToPath</code> in all four places, plus three guards: the computed path must land beside <code>.env.example</code> ; no source file anywhere may use <code>import.meta.url).pathname</code> (verified to fire by reintroducing it); and the mechanism is pinned as an executable assertion. <code>/api/health</code> now names the file it read and distinguishes "not set" from "no <code>.env</code> found here".
LESSON	Narrowly: <code>URL.pathname</code> is not a filesystem path. Broadly: this is the fourth entry where a <i>default concealed a failure</i> (#17, #28, #31, #36). The fallback is reasonable, it matches what the file would have said, and so the system works while being misconfigured. It also survived every check here, twice — CI runs Linux, the author's machine runs Linux, and neither path has a space. A single-platform suite cannot assert cross-platform correctness, so the honest response is a ban on the idiom rather than a test pretending to cover it.

#37 HTTP 414 on every IAM role with a path

SYMPTOM	Against a real account, clicking IAM roles and instance profiles showed nothing and returned <code>414 URI Too Long</code> .
CAUSE	Fastify caps a route parameter at 100 characters by default. <code>/api/resources/:arn</code> carries an ARN, and AWS service-linked roles carry an IAM <i>path</i> — <code>...:role/aws-service-role/elasticloadbalancing.amazonaws.com/AWSServiceRoleForElasticLoadBalancing</code> is 120 characters — so the router refused it before any handler ran. The remediation route has the same shape, so "How to fix" was broken for exactly the admin roles it matters most for. 10 of 19 IAM roles and profiles in the reporting account were over the cap , the longest at 145.
WHY NOTHING CAUGHT IT	Two reasons. Every fixture role had a short, path-less name (longest ARN 61 encoded), so the mock could not express the shape — the third time the fixture shared the code's blind spot (#29, #30). And <code>server.ts</code> called <code>listen()</code> at module scope, so importing it bound a port: no test could issue an HTTP request, and the router had no test surface at all.
FIX	<code>maxParamLength: 2048</code> , reasoned against IAM's documented maxima rather than picked round; <code>buildApp()</code> extracted so the HTTP layer is injectable; and a service-linked role added to the fixture, named in the answer key so deleting it fails a test.
LESSON	My first version of the regression check asserted the <i>encoded</i> length, and one case passed even with the bug restored — Fastify measures the <i>decoded</i> parameter. The attempt before that used a <code>sed</code> whose indentation did not match, so the "broken" run was actually the fixed code. Verifying that a guard fires is itself an operation that can silently fail, and the failure looks exactly like success. #23 and #27 are about guards that could not fire; this is about a check on a guard that could not fire. Make it fail on purpose and read the actual output, not the exit code.

9 Recurring failure patterns

Thirty-seven entries reduce to about five shapes. If you are asked "what did you learn building this?", this section is the answer — it is more convincing than any individual bug.

1. Two sources of truth for the same fact

Bugs #2, #17 and #31 are one bug three times: the endpoint calls go to, and the account id results are labelled with, were independently configurable with nothing asserting they agree. The output each time is a fabricated inventory of a real account. **Rule:** when two configuration values must be consistent, compute one from the other or refuse — never document the requirement and hope.

1b. A default that conceals a failure

Bugs #17, #28, #31 and #36. Each time a fallback was reasonable, matched what the missing configuration would have said, and therefore let the system run while misconfigured — scanning the mock under a real account's name, building AWS clients with an empty region, or ignoring a `.env` that was never read. The symptom always surfaces somewhere unrelated, or not at all. **Rule:** a default that silently substitutes for configuration the user believes they supplied is a trap. Prefer failing, or say loudly which source actually won.

2. "Absent" and "empty" are different facts

Bugs #11, #28 and #30. An empty string is a *present* value to any check written against `null`, and the AWS SDK is inconsistent about which one it sends — between services, and between real AWS and moto. **Rule:** normalise at the boundary where data enters, once.

3. A check that cannot fire is worse than no check

Bugs #23 and #27. A guard written `!isMock` instead of `!isMock()` is always false. A gitleaks rule with a lookahead panics and scans nothing. Both leave a green tick next to an absent control, and green is trusted. **Rule:** test that your guard fires, and pair a probabilistic check with a deterministic one.

4. A fixture that shares the code's blind spot proves nothing

Bugs #29, #30 and #37. The mock account was built to be adversarial about everything I thought of — and contained no IAM users, no answer-key entry for the orphaned elastic IP, and no role with an IAM path. All three passed every test while being wrong, and the third left more than half the IAM section of the product unreachable in a real account. **Rule:** a green suite over a fixture you also wrote is evidence about consistency, not correctness. Point the thing at reality and disbelieve a clean result.

5. Explanatory code fails silently

Bugs #19, #20, #22 and #28. A template header, a deploy command, an error message — nothing downstream consumes them, so they stay confidently wrong. A wrong ARN in a code path throws; a wrong ARN in an instruction gets pasted into someone's AWS account. **Rule:** make the thing that explains also be the thing that computes.

AND ONE ABOUT PROCESS

Five of the last six bugs were found by *using the product* — deploying the template for real, following the guide by hand, scanning a real account, and auditing the README against the running system. None was reachable from the test suite, because each lived in a seam between components or outside the repository entirely. The tests were necessary and were never going to be sufficient.

10 Limits, and what comes next

What breaks first on a large account, in order

1. **The graph rebuild, at roughly 50k resources.** Neo4j is rebuilt wholesale in one transaction — simple, leaves no stale nodes, and $O(\text{all resources})$ per scan. *Fix:* diff the Postgres snapshots, which already exist, and `MERGE` only what changed. The snapshots were designed with this in mind.
2. **Scan wall-clock across many regions.** 6 services $\times$ 30 regions is 180 units at a concurrency of 6, and per-bucket S3 work is four API calls each — so 10,000 buckets is 40,000 calls in one unit. Partly addressed by the Resource Explorer fast path. *Remaining fix:* per-service concurrency and splitting oversized units.
3. **Throttling, well before that.** `retryMode: adaptive` handles bursts, but a full parallel scan competes with the customer's own workloads. *Fix:* a token bucket per (service, region) sized from published quotas, and a scan budget the customer controls.
4. **The frontend, at about 2,000 nodes.** React Flow renders every node; dagre is $O(V+E)$ but the DOM is not. *Fix:* server-side aggregation — collapse a VPC to one node until expanded — and viewport virtualisation.
5. **The agent's context on broad questions.** *Fix:* tools should return aggregates with drill-down rather than rows, and say so when truncating.

What does not break: partial-failure handling and credential renewal both get *more* useful at scale, which is why they were built in from the start.

Multi-tenancy is the other axis. Today one module-level flag tracks whether a scan is running and the graph holds one account. Multi-tenant needs an account id on every node and query, per-tenant credential caching, and a job queue instead of an in-process scan. The storage model already carries `accountId` on every resource; the scan orchestration is what would change.

Deliberate gaps, and why each is a boundary rather than an omission

Gap	Why it is deliberate
CloudWatch-based idle detection	Current findings use structural signals only. "This volume is unattached" is defensible from one inventory pass; "this instance has been under 2% CPU for thirty days" is far more useful and needs metrics over time. The role already grants the permissions.
NACLs, peering, Transit Gateway	Reachability is conservative: it can <i>miss</i> a path, but a path it reports is justified by rules that really exist. Being conservative in the safe direction was the deliberate choice; peering and Transit Gateway are the biggest honest gaps.
An LLM judge over the evals	The named limitation of tier 2 — an answer that cites the right resources and describes them wrongly would pass. Lexical patterns also cannot test a negated claim at all.
Incremental graph updates	The first scaling limit, and the enabler for continuous rather than on-demand scans.
Neo4j read-only RBAC	Not available in Community. Enforced at the driver instead, and stated as such.

The next week, in priority order

1. **Incremental graph updates.** Removes the first scaling limit and makes scans cheap enough to run continuously.
2. **Real idle detection** from CloudWatch and Cost Explorer.
3. **An LLM judge** over a larger eval set, closing the named gap.
4. **Change detection as a first-class feature.** Diffing exists and the agent can query it; "what changed since yesterday, and does any of it matter?" is the question that makes this a product someone opens daily.
5. **More of the account:** ELB, ECS, EKS, API Gateway, CloudFront, Route 53. The collector interface is deliberately small — each is an afternoon — and load balancers in particular would fill a real gap in the reachability graph.
6. **NACLs, peering and Transit Gateway** in the reachability model.

11 Interview preparation

Answers grounded in things the code actually does. Where an answer concedes something, the concession is the strongest part — say it before you are asked.

Q Why two databases? Isn't that over-engineering for this scale?

At this scale, yes, if they were peers. They are not: Postgres is authoritative and Neo4j is a projection that rebuilds from it in one transaction with no AWS calls. That hierarchy is what makes it cheap — the graph is disposable, so the model I understood least well at the start is the one I can change freely.

Neo4j is there for one specific reason: "what can reach the production database?" is a variable-length path query. One `MATCH` with a `*1..n` hop in Cypher, a recursive CTE over a junction table in SQL. And scan history, partial-failure records and agent traces all fit badly in a graph, so Postgres was never optional.

Q Why not let the model write Cypher? It would be far more flexible.

Four reasons, and the fourth is the one that decided it. Safety: a curated tool cannot express a mutation, so the brief's hard rule becomes structural rather than asserted. Correctness: hand-written Cypher for reachability is reviewable and identical every run. Cost: a tool call returns rows; text-to-Cypher tends to return a schema, a failed query, an error and a retry. **Auditability:** because every tool records exactly which ARNs it returned, I can validate citations mechanically — which is impossible if the model invents the query.

There is still a `graph_query` escape hatch for novel questions, behind a write-clause validator and a read transaction, because neither layer is trusted alone.

Q How do you know the agent isn't making things up?

I do not trust it not to; I detect it when it does. Every tool records the ARNs it returned, every ARN in an answer is checked against that set, and anything unsupported is flagged on the response and shown to the user. Any unsupported citation fails an eval case outright.

It has fired for real. In a run two days ago the agent wrote `arn:aws:ec2:...:instance/` — a truncated placeholder — and the validator caught it. The root cause was mine: the tool did not return the resource its own question was about, so the correct ARN was genuinely unavailable. That is the sort of bug you only find if the check is mechanical.

Q You changed the IAM role we gave you. Why?

Because `ReadOnlyAccess` is not read-only in the sense an inventory product needs. It grants about 7,000 actions including `s3:GetObject`, `secretsmanager:GetSecretValue` and `lambda:GetFunction`, which returns a presigned URL to function source. dave.io needs to know a bucket exists, who can reach it, and how it is configured — never to read an object out of it. Granting that turns a compromise of dave.io's platform account into a compromise of every customer's data.

The detail I would raise with a real customer is `sqs:ReceiveMessage`: it is not read-only even literally, because receiving a message starts its visibility timeout and can hide it from the consumer that should have processed it. A scanner with that permission can breach your own hard rule through a permission nobody thinks of as a write.

I also scoped the trust policy to the scanner principal rather than `:root` — which does not mean the root user, it means every principal in the account — and added `sts:SourceIdentity` so customers can attribute scans in their own CloudTrail. The original template is kept alongside for comparison.

Q What's the weakest part of this?

The eval suite's blind spot, and I would rather name it than have it found. An answer that cites exactly the right resources and describes them wrongly passes. The `mustMention` patterns cover the cases where that has teeth, but lexical patterns cannot test a negated claim at all — I tried twice on one case and both attempts failed *correct* answers, the second on the sentence "a different kind of risk than an unused admin role", which states the exact distinction I was testing for. Closing it needs an LLM judge, which is the third item on my list.

Second: reachability ignores NACLs, peering and Transit Gateway. It is conservative — it can miss a path, but a path it reports is justified by rules that really exist — and I would rather under-report than fabricate. Still a real gap.

Q What went wrong while you built it?

Thirty-seven logged problems, and they reduce to about five shapes. The one I would lead with: three separate times I built a route to the same catastrophic output — a complete, confident, entirely fictional inventory of a real AWS account. Once from a leaked `AWS_ENDPOINT_URL` that the SDK reads directly from the environment, once from moto's account namespacing, once from a mode override honouring a real account's role ARN while talking to the mock. Same cause every time: two sources of truth for "which account am I looking at?" with nothing asserting they agree.

The general lesson I took: when two configuration values must be consistent, do not document the requirement — compute one from the other, or refuse. The third fix is a pure function with ten tests, because the condition was load-bearing and untestable inside a module that reads the environment at import.

Q Your tests all pass. How much does that actually tell me?

Less than it looks, and I can prove it. A hundred and fifty-seven tests passed while the admin analysis silently ignored IAM users entirely — because the mock account I wrote contained no IAM users, so the fixture shared the code's blind spot. I found it by pointing the scanner at a real account and disbelieving an empty findings panel.

The same thing happened with an idle-detection rule that was correct and never fired once: the answer key had the same omission as the collector. So the tests are necessary and were never going to be sufficient, and the fixture being adversarial about everything I thought of is exactly the limitation — it cannot be adversarial about what I did not.

Q Why is there no "connect my AWS account" form in the UI?

Because this API has no authentication. That form would be an unauthenticated endpoint that assumes a role into somebody's AWS account and stores a value the role template itself calls a credential. Building it would have made the demo look more complete and the product less defensible.

So the UI guides onboarding instead of performing it: it generates an ExternalId, renders the exact CloudFormation command pre-filled with the principal to trust, and tests the connection that is already configured — diagnosing failures rather than echoing SDK errors. A multi-tenant version would put authentication, per-tenant isolation and encrypted secret storage in first, and only then offer the form.

Q A bucket has Block Public Access turned off. Is it public?

No — and this is the distinction I care most about in the project. Turning the block off grants nobody anything; it removes the guardrail that would neutralise a permissive policy. An anonymous request still gets 403 unless a policy or ACL actually grants access.

But "not public" is an unsatisfying answer to someone who has just weakened their own defences, so they are two separate verdicts: `isPublic` means anonymous access is granted now, `isUnprotected` means nothing would stop it being granted. Separate tabs, separate agent tools, and a paragraph of the system prompt — because the model conflates them too.

Q How would you make this production-ready?

In order: incremental graph updates from the Postgres snapshots, which removes the first scaling limit and makes continuous scanning affordable. Per-service rate limiting with a customer-controlled scan budget, because competing with a customer's own workloads is worse than being slow. Multi-tenancy — account id on every query, per-tenant credential caching, and a job queue instead of an in-process scan; the storage model already carries `accountId`, so it is the orchestration that changes. Then Neo4j Enterprise or a read replica for genuine database-level read-only, which Community cannot give me.